



Sunbeam Institute of Information Technology
Pune and Karad

Module - Concepts of Operating Systems and Administration

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

- **Storage** is used to store data permanently or temporarily in a computer system.
- Main purpose of **Storage** is to
 - Store OS (Linux, Windows)
 - Store user data (files, videos, code)
 - Store applications
 - Store logs and system data

Feature	Primary	Secondary
Speed	Very fast	Slower than RAM
Volatility	Volatile	Non Volatile
Size	Limited	Large
Cost	Expensive	Cheaper

- **Types of Storage**

- **Primary storage (Temporary storage)**

- RAM
 - Cache
 - ROM ✂

- **Secondary storage (Permanent storage)**

- HDD (Hard Disk Drive)
 - Spinning magnetic disks, usually inexpensive, large, and relatively slow
 - SSD (Solid State Drive)
 - Flash memory storage, usually expensive, fast, small, and shorter-lived
 - USB drives
 - USB - Connection protocol for various external devices
 - Removable storage media, connected via USB
 - Removable storage disk, connected via USB

Types of Secondary Storage

- Based on technology used, secondary storage is classified into:
 - Magnetic Storage Devices
 - Stores data using magnetic fields on a rotating surface
 - Disk surface is coated with magnetic material
 - Eg. HDD (Hard Disk Drive), Floppy, ZipDisk
 - Electronic (Solid-State) Storage Devices
 - Stores data using electrical charges in semiconductor memory
 - No moving part, Uses flash memory (NAND)
 - Eg. SSD (SATA SSD), NVMe SSD, USB Flash Drive
 - Optical (Polycarbonate) Storage Devices
 - Uses laser light to read/write data on a polycarbonate disk
 - Polycarbonate plastic Coated with reflective layer
 - Laser beam scans disk surface
 - Eg. CD, DVD, Blu-Ray

Comparison of Secondary Storage

Feature	Magnetic	Electronic	Optical
Technology	Magnetic field	Electric charge	Laser
Speed	Slow	Fast	Very slow
Moving parts	Yes	No	No
Cost	Cheap	Expensive	Very cheap
Capacity	Very high	High	Low
Reliability	Medium	High	Low

Types of Storage Interfaces

- **PATA (IDE)**- Parallel ATA / IDE
 - Old interface used for HDDs and CD drives
 - Parallel, Slow (~133 MB/s), Wide ribbon cable, Max 2 devices per cable
 - Naming Conventions - /dev/hda, /dev/hdb
- **SATA** – Serial ATA
 - Modern replacement for PATA, used for HDD, SATA SSD
 - Serial, speed Up to ~600 MB/s, Thin and flexible Cable, One device per port
 - Naming convention - /dev/sda, /dev/sdb
- **SCSI** - Small Computer System Interface
 - Used in servers and enterprise systems
 - High speed, more reliable, Multiple device
 - Naming convention - /dev/sdX
- **SAS** - Serial Attached SCSI
 - Advanced version of SCSI, Dual port, Better Performance than SATA
 - High speed, very high reliability, used in enterprise servers, data centers, high end storage devices

Types of Storage Interfaces

- **USB - Universal Serial Bus**
 - External storage interface, highly portable, moderate speed, plug and play
 - Versions - USB 2.0(~480 Mbps), USB 3.0(~5 Gbps), USB 3.1/3.2(faster)
- **PCIe (NVMe) - NVMe Interface**
 - High-speed interface for modern SSDs
 - Very high speed (GB/s), Very Low Latency, Direct communication with CPU
 - Naming convention - /dev/nvme0n1
- **Linux naming depends on driver, not on interface**
- In Linux many devices appears **/dev/sdX**, Because Linux uses **SCSI layer abstraction**
 - **/dev/sda** - A hard disk that uses the SCSI driver.
 - **/dev/nvme0n1** - The first hard disk on an NVM Express (NVMe) interface.
 - **/dev/hda** - The (legacy) IDE disk device type.
 - **/dev/vda** - A disk in a KVM virtual machine that uses the virtio disk driver
 - **/dev/xvda** - A disk in a Xen virtual machine that uses the Xen virtual disk driver

- Storage drives are divided into partitions for more efficient and organized use
- **MBR**
 - The master boot record (MBR) is the first physical sector on a storage drive and a type of partition structure
 - The MBR boot sector contains the boot loader that loads the operating system into memory
 - It also contains the partition table of the storage drive
 - MBR determines what sectors are available to each partition, as well as which partitions are bootable and which are not
 - It has three major disadvantages:
 - The maximum storage space of an MBR-partitioned drive is two terabytes
 - MBR-partitioned drives can have a maximum of four primary partitions
 - The boot data is stored all in one sector, which increases the risk of corruption

- **GPT**
 - The GUID Partition Table (GPT) is a successor to MBR that makes up for the latter's shortcomings
 - Like MBR, it is a partition structure, but it employs a more modern design and is part of the UEFI standard
 - Every partition on a drive is assigned a globally unique identifier—a GUID—to distinguish it from every other partition on (theoretically) every drive
 - The storage space and partition number maximums are so large that they are not currently achievable, and any limitations are going to be imposed by the file system type or operating system kernel, rather than GPT itself.
 - GPT also has the advantage of storing its boot data in multiple locations on a drive to enhance redundancy
 - If the primary location is corrupted, GPT can leverage one of the other copies to restore the boot data

MBR = Master Boot Record

- Old partitioning scheme
- Introduced in early systems (BIOS-based)
- Structure of MBR
 - [Bootloader | Partition Table | Signature]
 - Stored in first sector (512 bytes) of disk
 - Contains: Bootloader, Partition table
- Limitations
 - Max disk size: 2 TB
 - Max partitions: 4 primary partitions
 - To create more: Use extended + logical partitions
- BIOS boot
- Bootloader in MBR

GPT = GUID Partition Table

- Modern partitioning scheme
- Part of UEFI system
- Structure of GPT
 - [Protective MBR | GPT Header | Partition Entries | Backup Header]
- Supports large disks (>2 TB)
- Stores multiple copies of partition table (backup)
- Up to 128 partitions (or more)
- Better reliability (backup + CRC checks)
- UEFI boot
- Bootloader in EFI partition

Feature	MBR	GPT
Full Form	Master Boot Record	GUID Partition Table
Age	Old	Modern
Max Disk Size	2 TB	> 2 TB
Partitions	4 primary	128+
Boot Mode	BIOS	UEFI
Backup	No	Yes
Reliability	Low	High
Partition Info	Single location	Multiple copies

Partitioning command

Command

Use

`fdisk`

Basic partitioning (MBR)

`parted`

Advanced (GPT, large disks)

`lsblk`

View disks

`blkid`

FS info

`partprobe`

Reload changes

`cfdisk`

UI-based

`sfdisk`

Scripting

- **fdisk (Most Common - MBR)**

- Command > **fdisk /dev/sda**

- **Options**

- m - Help
- p - Print partition table
- n - New partition
- d - Delete partition
- t - Change partition type
- w - Write changes (save)
- q - Quit without saving
- l - List partition types

- **parted (Advanced - GPT Support)**

- Command > **parted /dev/sda**

- **Commands**

- print - Show partitions
- mklabel gpt - Create GPT table
- mkpart - Create partition
- rm - Delete partition
- quit - Exit

- After creating the partitions on a new drive, you must add a filesystem to organize the storage of the actual data
- Different filesystems have different benefits, and there are many of them
- In general, most Linux distributions use either the ext4 or XFS filesystems.
- **ext4 Filesystem**
 - The ext4 filesystem is the one of the oldest file systems
 - It provides journaling and is recognized by most distributions.
 - To place the ext4 filesystem on the second partition on the new disk, type:
 - **mkfs.ext4 <partition>**
 - The mkfs also recognizes a slightly different syntax.
 - **mkfs -t ext4 <partition>**
- **XFS Filesystem**
 - While the ext4 filesystem has been the default for many Linux distributions, it is very common to find newer systems relying on XFS
 - XFS has many modern advantages over ext4 and relatively few disadvantages
 - XFS recognizes a larger partition size
 - XFS recognizes a larger file size
 - XFS avoids inode exhaustion

- Now that you have installed the drive, created the partitions, and placed the filesystem, the storage capacity is ready for use
- **mount**
 - The term for attaching storage to the FHS is referred to as "mounting," and the command is mount
 - To mount the partition, first create a directory
 - The directory is referred to as a mount point
 - Use the mount command to mount a fs to a directory
 - > **sudo mount <partition> <mount point>**
- **findmnt**
 - Find all the mounted file system
- **umount**
 - The logical reverse of this is to detach or unmount the storage area, and the related command is **umount**
 - Use umount command to unmount the mounted partition
 - > **umount <partition>**

Persistent Mounting

- /etc/fstab is the main configuration file to persistently mount partitions
- /etc/fstab content is used to generate system mounts by the system-fstab-generator utility

```
/dev/disk/by-uuid/25fd6706-979a-4e16-b52d-8af2a2331d5f /home ext4 defaults 0 0
```

- Every entry contains:
 - Column 1: device to be mounted
 - Column 2: mount point
 - Column 3: file system
 - Column 4: extra mount options
 - Column 5 and 6: always 0
- To update systemd, make sure to use systemctl daemon-reload after editing /etc/fstab
- Use mount -a to check the entries are working from fstab file
- Note: Do not persist mount on /mnt as it is used to mount a FS temporarily

- **Logical Volume Manager**

- It is a storage management layer in Linux that sits between Physical Disks and Filesystem
- Instead of using disks/partitions directly, LVM creates a virtual layer:
 - Disk → LVM → Logical Volume → Filesystem

- Traditional partitioning:

- Disk → Partition → Filesystem
- Limitations:
 - Fixed size partitions
 - Difficult to resize
 - Cannot combine multiple disks easily

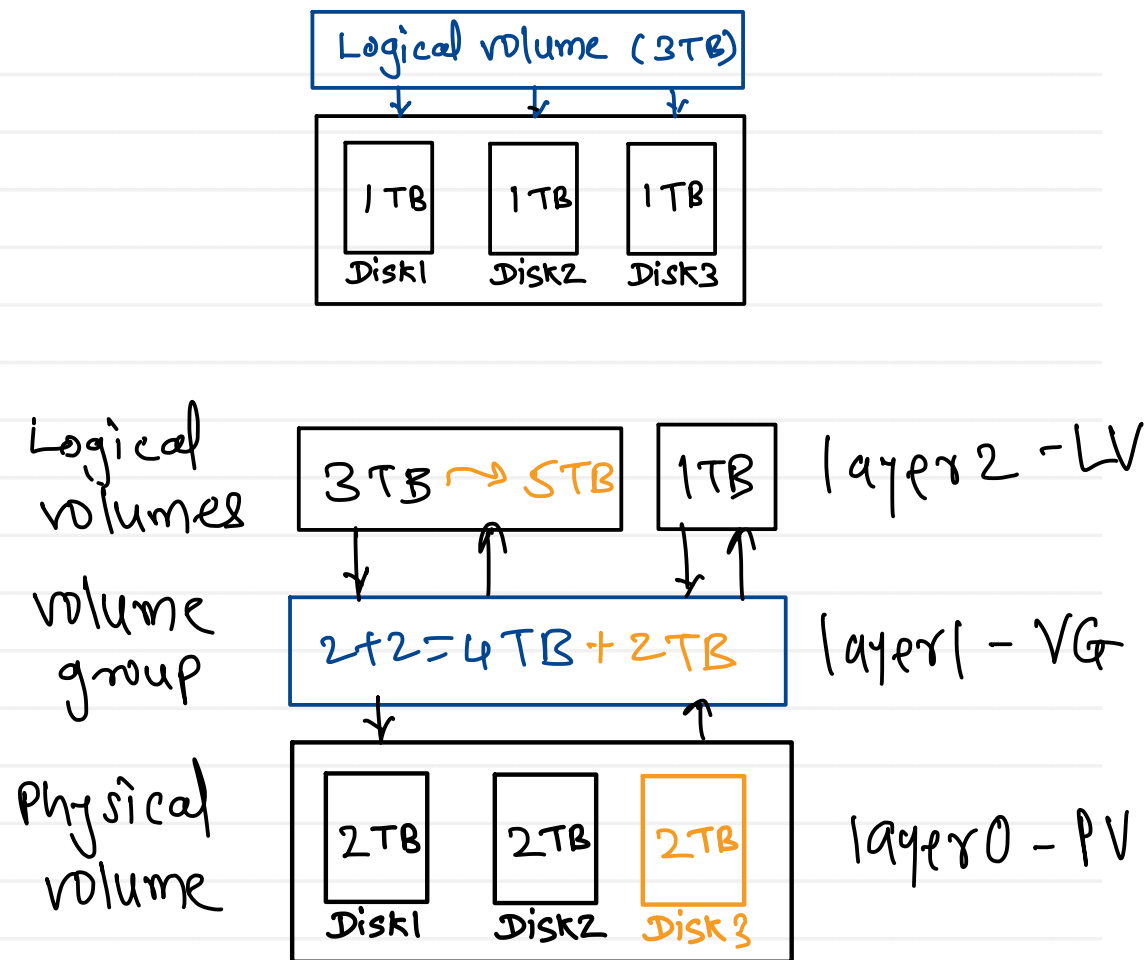
- Solution → LVM

- Resize partitions dynamically
- Combine multiple disks
- Better storage utilization

LVM Provides

- Flexibility
 - Increase size anytime
 - Reduce size (carefully)
- Scalability
 - Add new disk → extend storage
- Abstraction
 - User does NOT deal with raw disks directly

- Logical Volume Manager functions based on three configuration layers for storage
- The first layer is one or more **Physical Volumes (PV)**
 - PVs are drives or partitions allocated to LVM
- The next layer represents the aggregated storage capacity of any designated PVs that are added to the **Volume Group (VG)**
- The top layer consists of one or more **Logical Volumes (LV)**, which are treated by the system as if they were standard partitions
 - In reality, they are carved space from VGs that may store data on multiple PVs
- The /dev/mapper/ directory contains all of the logical volumes on the system that are managed by LVM
 - /dev/mapper/<volume group name>-<logical volume name>





LVM Advantages

- Dynamically create, delete, and resize volumes without having to reboot the system
- Day-to-day management of volumes is easier once everything is set up
- Map multiple logical volumes across multiple physical devices
- A logical volume can exceed the size of any one physical device (as long as it doesn't exceed the total size of devices in the volume group)
- Create virtual snapshots of each logical volume so you can quickly and easily revert a volume to a specific state

- **Physical Volume**

- ✓ pvcreate: creates new physical volume
- pvscan: scans existing physical volumes
- ✓ pvdisplay: displays attributes of selected physical volume
- pvs: shows existing physical volumes

- **Volume Group**

- ✓ vgcreate: creates new volume group
- vgscan: scans existing volume groups
- ✓ vgdisplay: displays attributes of selected volume group
- vgs: shows existing volume groups

- **Logical Volume**

- ✓ lvcreate: creates new logical volumes
- lvscan: scans existing logical volumes
- ✓ lvdisplay: displays attributes existing logical volumes
- lvvs: shows existing logical volumes

- The processor checks for the BIOS/UEFI firmware and executes it
- BIOS/UEFI checks for bootable media from internal storage devices or peripherals like USB flash drives and DVD-ROMs. It locates a valid device to boot the system
- BIOS/UEFI loads the primary boot loader (probably GRUB2) from the MBR/GPT partition into memory. It also loads the partition table along with it.
- GRUB2 prompts the user to select an operating system to boot. If the user does not respond, then the default operating system is booted.
- The boot loader determines the kernel and locates the corresponding kernel binary
- It then uploads the respective initrd image into memory and transfers control of the boot process to the kernel
- The kernel configures the available hardware drivers, including processors, I/O subsystems, and storage devices. It decompresses the initrd image and mounts it to load the necessary drivers. If the system implemented any virtual devices, such as LVM or software RAID, then they are initialized
- The kernel mounts the main root partition and releases unused memory
- The systemd program runs to set up the user environment. It becomes process ID 1



Linux Booting

- The systemd program searches for the default.target file, which contains details about the services to start
- It mounts the file system based on the /etc/fstab file or .mount files and begins the process of starting services
- On most systems, the selection is either multi-user.target or graphical.target
- If graphical mode is selected, then a display manager starts and the login window is displayed on the screen
- The user enters a user name and password to log in to the system
- The system authenticates the user. If the user is valid, then various profile files are executed
- The shell starts, and the system is ready for the user

